

SEG800 DIP Cheatsheet

Test 4: Spatial Filtering, Frequency-Domain Filtering, Color Fundamentals

1 Part I: Spatial Filtering

Noise model:

$$\hat{I}(i, j) = I(i, j) + n(i, j)$$

Gaussian noise: small random variations. Salt-and-pepper: extreme dark/bright spikes.

1.1 Linear Filtering

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

Kernel normalization for smoothing:

$$\sum_s \sum_t w(s, t) = 1$$

1.2 Correlation vs Convolution

$$\text{corr: } (w \circledast f)(x, y) = \sum_s \sum_t w(s, t) f(x + s, y + t)$$

$$\text{conv: } (w \star f)(x, y) = \sum_s \sum_t w(s, t) f(x - s, y - t)$$

Convolution flips the kernel; correlation does not. Symmetric kernels give the same output. Same-size odd $m \times n$ filtering: zero-pad $\lfloor m/2 \rfloor, \lfloor n/2 \rfloor$ each side; for 3×3 , pad 1.

1.3 Core Kernels

Box:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Bilinear:

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Gaussian:

$$G(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Worksheet Q4 template with $\sigma = 2$ and 3×3 ($x, y \in \{-1, 0, 1\}$): First compute the **unnormalized** entries:

$$G(-1, -1) = e^{-2/8} = 0.7788, \quad G(-1, 0) = e^{-1/8} = 0.8825, \quad G(0, 0) = 1$$

Use symmetry for the rest, so

$$G \approx \begin{bmatrix} 0.7788 & 0.8825 & 0.7788 \\ 0.8825 & 1.0000 & 0.8825 \\ 0.7788 & 0.8825 & 0.7788 \end{bmatrix}, \quad \sum G \approx 7.6452$$

Then **normalize** by dividing every entry by the sum:

$$h(x, y) = \frac{G(x, y)}{\sum_{x, y} G(x, y)}$$

$$h = \frac{1}{7.6452} \begin{bmatrix} 0.7788 & 0.8825 & 0.7788 \\ 0.8825 & 1.0000 & 0.8825 \\ 0.7788 & 0.8825 & 0.7788 \end{bmatrix} \approx \begin{bmatrix} 0.1019 & 0.1154 & 0.1019 \\ 0.1154 & 0.1308 & 0.1154 \\ 0.1019 & 0.1154 & 0.1019 \end{bmatrix}$$

1.4 Median Filter

For a 3×3 median filter: collect 9 values, sort, choose the 5th. Best for salt-and-pepper noise; usually preserves edges better than averaging.

1.5 Separable Kernels

If

$$w(x, y) = w_1(x)w_2(y)$$

then filter in two 1-D passes. Proof idea: $(w_1 w_2) * f = w_1 * (w_2 * f) = w_2 * (w_1 * f)$. Cost comparison for image $M \times N$, kernel $m \times n$:

$$\text{direct} = MNmn, \quad \text{separable} = MN(m + n)$$

Speed-up:

$$\frac{MNmn}{MN(m + n)} = \frac{mn}{m + n}$$

Square $m \times m$ case:

$$O(MNm^2) \rightarrow O(MNm)$$

For 5×5 : $25 \rightarrow 10$ ops/pixel, speed-up = $2.5\times$. Worksheet factorization example:

$$w = \begin{bmatrix} 1 & 3 & 1 \\ 2 & 6 & 2 \end{bmatrix}$$

Pick $(2, 1)$, so $E = 2$,

$$\mathbf{c} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{r} = \begin{bmatrix} 2 & 6 & 2 \end{bmatrix}, \quad w_1 = \mathbf{c}, \quad w_2 = \frac{\mathbf{r}}{E} = \begin{bmatrix} 1 & 3 & 1 \end{bmatrix}$$

Check:

$$w_1 w_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & 3 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 & 1 \\ 2 & 6 & 2 \end{bmatrix}$$

1.6 Gaussian Pixel Template

For a target pixel, use its local neighborhood and compute

$$g(i, j) = \sum_x \sum_y h(x, y) f(i + x, j + y)$$

Worksheet image at $(2, 3)$ uses neighborhood

$$\begin{bmatrix} 100 & 102 & 98 \\ 101 & 99 & 103 \\ 97 & 100 & 102 \end{bmatrix}$$

so

$$\begin{aligned} g(2, 3) &\approx 100(0.1019) + 102(0.1154) + 98(0.1019) \\ &\quad + 101(0.1154) + 99(0.1308) + 103(0.1154) \\ &\quad + 97(0.1019) + 100(0.1154) + 102(0.1019) \end{aligned}$$

$$g(2, 3) \approx 100.26 \approx 100$$

For high-boost, first compute $\bar{f}(i, j)$ this way, then use $g_{\text{mask}} = f - \bar{f}$.

1.7 Sharpening

Laplacian:

$$\nabla^2 f = f(x + 1, y) + f(x - 1, y) + f(x, y + 1) + f(x, y - 1) - 4f(x, y)$$

Common kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Sharpening:

$$g = f + c\nabla^2 f$$

Choose sign of c to match the kernel convention.

Unsharp mask and high-boost:

$$g_{\text{mask}} = f - \bar{f}$$

$$g = f + k g_{\text{mask}} = (1 + k)f - k\bar{f}$$

$k = 1$: unsharp masking. $k > 1$: high-boost.

2 Part II: Frequency-Domain Filtering

2.1 2-D DFT / IDFT

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(ux/M + vy/N)}$$

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(ux/M + vy/N)}$$

Magnitude, phase, power:

$$F = R + jI = |F|e^{j\phi}$$

$$|F| = \sqrt{R^2 + I^2}, \quad \phi = \text{atan2}(I, R), \quad P = |F|^2$$

DC component:

$$F(0, 0) = MN\bar{f}$$

2.2 Sampling and Aliasing

1-D Nyquist:

$$f_s > 2f_{\text{max}}$$

2-D Nyquist:

$$\frac{1}{\Delta T} > 2\mu_{\text{max}}, \quad \frac{1}{\Delta Z} > 2\nu_{\text{max}}$$

Undersampling causes aliasing and moire patterns.

2.3 Translation and Centering

Spatial shift changes phase, not magnitude:

$$f(x - x_0, y - y_0) \iff F(u, v)e^{-j2\pi(u x_0/M + v y_0/N)}$$

Center spectrum before filtering:

$$f(x, y)(-1)^{x+y} \iff F\left(u - \frac{M}{2}, v - \frac{N}{2}\right)$$

2.4 Convolution Theorem and Padding

$$f * h \iff FH$$

DFT multiplication gives circular convolution unless padded enough. For $A \times B$ convolved with $C \times D$:

$$P \geq A + C - 1, \quad Q \geq B + D - 1$$

Equal-size shortcut:

$$P \geq 2M - 1, \quad Q \geq 2N - 1$$

Common convenient choice:

$$P = 2M, \quad Q = 2N$$

Butterworth/other radial filters are built on the padded grid $P \times Q$, centered at $(P/2, Q/2)$. In this course, if not stated otherwise, usually use $P = 2M$, $Q = 2N$ before computing $D(u, v)$.

2.5 Frequency-Domain Filtering Workflow

1. Pad image to $P \times Q$.
2. Multiply by $(-1)^{x+y}$.
3. Compute DFT $F(u, v)$.
4. Build centered filter $H(u, v)$.
5. Multiply: $G(u, v) = H(u, v)F(u, v)$.
6. IDFT, multiply by $(-1)^{x+y}$, take real part, crop.

2.6 Radial Distance and Filter Families

$$D(u, v) = \sqrt{\left(u - \frac{P}{2}\right)^2 + \left(v - \frac{Q}{2}\right)^2}$$

Ideal LPF:

$$H_{ILPF} = \begin{cases} 1, & D \leq D_0 \\ 0, & D > D_0 \end{cases}$$

Gaussian LPF:

$$H_{GLPF} = e^{-D^2/(2D_0^2)}$$

Butterworth LPF:

$$H_{BLPF} = \frac{1}{1 + (D/D_0)^{2n}}$$

Worksheet template: find padded center $\rightarrow$ compute $D(u, v) \rightarrow$ substitute into H_{BLPF} . High-pass from low-pass:

$$H_{HP} = 1 - H_{LP}$$

Gaussian HPF:

$$H_{GHPF} = 1 - e^{-D^2/(2D_0^2)}$$

Butterworth HPF:

$$H_{BHPF} = \frac{1}{1 + (D_0/D)^{2n}}, \quad H_{BHPF}(0, 0) = 0$$

2.7 Behavior and Computation

Small D_0 in LPF $\Rightarrow$ stronger smoothing. Larger D_0 in HPF $\Rightarrow$ stronger removal of low frequencies.

Ideal filters have strongest ringing. Gaussian has least ringing. Butterworth is a tunable compromise via order n .

Ops estimate:

$$\text{spatial} = MNmn, \quad \text{FFT} \approx 2MN \log_2(MN)$$

Use FFT when kernels are large and nonseparable.

3 Part III: Color Fundamentals

3.1 Terms and Primaries

Radiance = physical energy, luminance = perceived light, brightness = subjective, intensity = numeric proxy.

Additive light primaries: RGB.

$$R + G = Y, \quad G + B = C, \quad R + B = M, \quad R + G + B = W$$

Subtractive primaries: CMY. Printing uses CMYK.

3.2 Chromaticity and Gamut

$$x = \frac{X}{X + Y + Z}, \quad y = \frac{Y}{X + Y + Z}, \quad z = \frac{Z}{X + Y + Z}, \quad x + y + z = 1$$

Gamut = set of colors a device can reproduce.

3.3 RGB, CMY, CMYK

Normalized RGB cube:

$$0 \leq R, G, B \leq 1, \quad R = G = B \Rightarrow \text{gray}$$

$$(0, 0, 0) = \text{black}, \quad (1, 1, 1) = \text{white}$$

24-bit RGB gives $256^3 = 2^{24} = 16,777,216$ colors.

RGB $\leftrightarrow$ CMY:

$$C = 1 - R, \quad M = 1 - G, \quad Y = 1 - B$$

$$R = 1 - C, \quad G = 1 - M, \quad B = 1 - Y$$

CMYK from CMY:

$$K = \min(C, M, Y)$$

If $K \neq 1$:

$$C' = \frac{C - K}{1 - K}, \quad M' = \frac{M - K}{1 - K}, \quad Y' = \frac{Y - K}{1 - K}$$

3.4 HSI

Hue = basic color, saturation = purity, intensity = brightness.

$$I = \frac{R + G + B}{3}$$

$$S = 1 - \frac{3}{R + G + B} \min(R, G, B)$$

$$\theta = \cos^{-1} \left(\frac{\frac{1}{2}[(R - G) + (R - B)]}{\sqrt{(R - G)^2 + (R - B)(G - B)}} \right)$$

$$H = \begin{cases} \theta, & B \leq G \\ 360^\circ - \theta, & B > G \end{cases}$$

Hue sectors:

$$0^\circ \leq H < 120^\circ : RG, \quad 120^\circ \leq H < 240^\circ : GB, \quad 240^\circ \leq H < 360^\circ : BR$$

3.5 Color Processing Rules

Two main approaches:

- Process each RGB channel and recombine.
- Convert to HSI and process only I when the task is brightness, smoothing, sharpening, or histogram equalization.

Do not histogram-equalize RGB channels independently: different nonlinear mappings can distort hue.

Brightness scaling:

$$R' = kR, \quad G' = kG, \quad B' = kB$$

or in HSI:

$$I' = kI$$

RGB complement:

$$R' = 1 - R, \quad G' = 1 - G, \quad B' = 1 - B$$

3.6 Pseudocolor and Slicing

Intensity slicing:

$$\mathbf{g}(x, y) = \mathbf{c}_k \quad \text{if } l_{k-1} \leq f(x, y) < l_k$$

Intensity-to-color mapping:

$$R = T_R(f), \quad G = T_G(f), \quad B = T_B(f)$$

Color distance in RGB:

$$D = \sqrt{(R - a_R)^2 + (G - a_G)^2 + (B - a_B)^2}$$

for color slicing around a target color.